

9/10/26

PROBLEM 1)

Part (a)

void f1(int n)

```

{
  int i = 2;
  while (i <= n) {
    /* O(1) */
    i = i * i;
  }
}

```

loop terminates when $i > n$ is true. Using k to represent the number of while loop iterations,

• $k = 0 : i = 2$ • $k = 1 : i = 4$ • $k = 2 : i = 16$ • $k = 3 : i = 256$ • $k = 4 : i = 65536$

$$T(n) = \sum_{k=1}^{k_{\max}} \Theta(1)$$

The relationship between i and k is $i_k = 2^{(2^k)}$. The while loop's termination condition can be written as $2^{2^k} \geq n$

$$2^k \geq \log_2 n$$

$$k \geq \log_2 n (\log_2 n)$$

$$\therefore k_{\max} = \log_2 n \log_2 n$$

$$\therefore T(n) = \sum_{k=1}^{\log_2 n \log_2 n} \Theta(1) = \boxed{\Theta(\log \log n)}$$

Part (b)

void f2(int n)

```

{
  for (int i = 1; i <= n; i++) {
    if ((i % (int) sqrt(n)) == 0) {
      for (int k = 0; k < pow(i, 3); k++) {
        /* O(1) */
      }
    }
  }
}

```

loop terminates when $i > n$ is true

$$T(n) = \sum_{i=1}^n \left(\Theta(1) + \left[\begin{array}{l} \text{inner loop cost if} \\ \text{the if condition} \\ \text{passes} \end{array} \right] \right)$$

finding the i 's that trigger this
 $i \% \sqrt{n} = 0$ corresponds to i being a multiple of $\sqrt{n}$ ($i = j\sqrt{n}$)

$$\therefore T(n) = \sum_{i=1}^n \Theta(1) + \sum_{j=1}^{\sqrt{n}} \sum_{k=0}^{(j\sqrt{n})^3 - 1} \Theta(1)$$

$$\sum_{k=0}^{(j\sqrt{n})^3 - 1} \Theta(1) = \Theta(j(\sqrt{n})^3) = \Theta(j^3) \cdot \Theta(n^{3/2})$$

$$\sum_{j=1}^{\sqrt{n}} \Theta(j^3) \cdot \Theta(n^{3/2}) = \Theta(n^{3/2}) \cdot \sum_{j=1}^{\sqrt{n}} j^3$$

* fixed power sum grows one degree higher

$$= \Theta(n^{3/2}) \cdot \Theta((\sqrt{n})^4)$$

$$= \Theta(n^{3/2}) \cdot \Theta(n^2)$$

$$\boxed{T(n) = \Theta(n^{7/2})}$$

part (c)

```

each  $\Theta(1)$  [ for (int i=1; i <= n; i++) {
            for (int k=1; k <= n; k++) {
                if (A[k] == i) {
                    for (int m=1; m <= n; m=m+m) {
                        // O(1)
                    }
                }
            }
        }
    ]
    
```

$$T(n) = \sum_{i=1}^n \sum_{k=1}^n (\Theta(1) + \text{inner loop cost, only if } A[k] == i)$$

innermost loop cost } doubling input size (N) adds exactly one extra iteration in an algorithm with a $O(\log n)$ runtime.

of times it actually triggers } for each fixed k, across all n values of i, the if statement will fire at most once $\rightarrow$ there are n values of k.

$$\sum_{k=1}^n \Theta(\log n) = n \cdot \Theta(\log n) = \Theta(n \log n)$$

$$\therefore T(n) = \sum_{i=1}^n \sum_{k=1}^n (\Theta(1) + \Theta(n \log n)) = \Theta(n^2) + \Theta(n \log n)$$

since $\Theta(n^2) > \Theta(n \log n)$

$$T(n) = \Theta(n^2)$$

part (d)

```

int f(int n)
{
    int *a = new int [10];
    int size = 10;
    for (int i=0; i < n; i++)
    {
        if (i == size)
        {
            int newSize = 3 * size / 2;
            int *b = new int [newSize];
            for (int j=0; j < size; j++) b[j] = a[j];
            delete [] a;
            a = b;
            size = newSize;
        }
        a[i] = i + i;
    }
}
    
```

$$T(n) = \sum_{i=0}^{n-1} (\Theta(1) + [\text{cost of resize (if } i == \text{size})])$$

of times this triggers :

$$\text{size}_0 = 10$$

$$\text{size}_1 = 10 \cdot \frac{3}{2}$$

$$\text{size}_2 = 10 \cdot \left(\frac{3}{2}\right)^2$$

$$\text{size}_x = 10 \cdot \left(\frac{3}{2}\right)^x$$

$\therefore$ a resize will happen whenever i catches up to the current size (i.e 10, 15, 22.5, etc...) meaning that the cost of a resize is $\Theta\left(\left(\frac{3}{2}\right)^x\right)$ so the TOTAL resize cost can be represented by $\sum_{k=1}^{\infty} \Theta\left(\left(\frac{3}{2}\right)^k\right)$

$$\Theta\left(\left(\frac{3}{2}\right)^x\right) = O(n)$$

$$\therefore T(n) = \sum_{i=0}^{n-1} \Theta(1) + \Theta(n) = \Theta(n) + \Theta(n) = \Theta(n)$$

where $\left(\frac{3}{2}\right)^x \approx n$

PROBLEM 2)

a) returned linked list when in1 = 1,2,3,4 & in2 = 5,6:

llrec(1,5) : 1 $\rightarrow$ next

llrec(5,2) : 5 $\rightarrow$ next

llrec(2,6) : 2 $\rightarrow$ next

llrec(6,3) : 6 $\rightarrow$ next

llrec(3,null) : in2 is null, so you return 3

$\therefore$ this recurses 5 times. Each call swaps which list goes first. Once in2 runs out, which occurs after 6, the base case occurs and the remaining tail of in1 (which is 3,4) doesn't change. Each node's next therefore gets wired to whatever was coming back below it, resulting in 1 $\rightarrow$ 5 $\rightarrow$ 2 $\rightarrow$ 6 $\rightarrow$ 3 $\rightarrow$ 4

b) returned linked list when in1 = nullptr & in2 = 2:

Since in1 = nullptr, the base case is hit in the very first call, specifically if (in1 == nullptr) { return in2; }. So no recursion occurs

and the linked list just returns the value of in2 which in this case is 2